Dynamic Path Offset Correction on UR10e via RTDE

Oct 28, 2026 · @Rasitha R

Overview

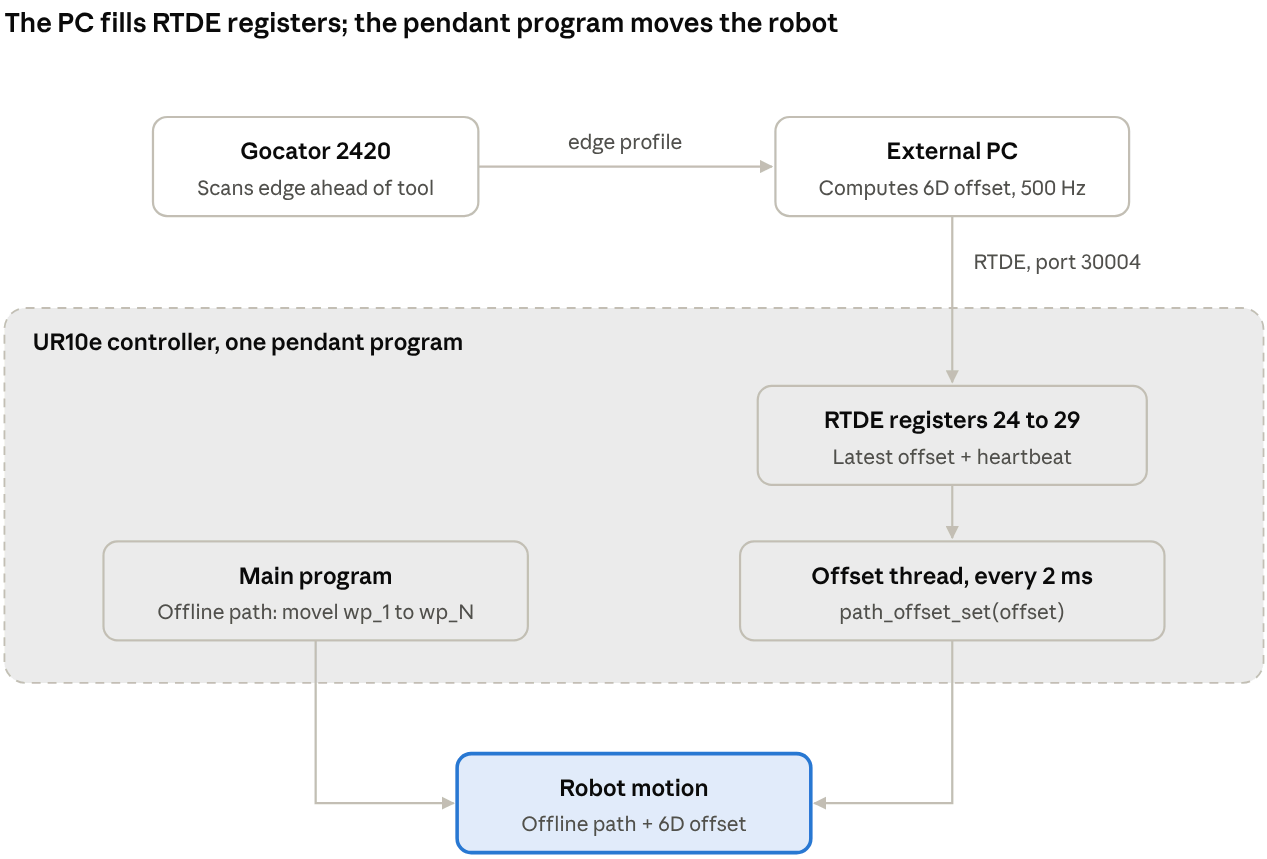
The offline path keeps running from the UR10e teach pendant, and an external PC sends a live 6D offset over RTDE at the same time. The robot controller adds the two together using UR's built-in Online Path Offset feature (`path_offset_*` URScript functions, available since PolyScope 5.6.0).

Goal: correct the robot's tool pose in 6D (X, Y, Z, Rx, Ry, Rz) while it moves, based on sensor data such as a Gocator 2420 edge scan, without stopping the program or regenerating the path.

Key idea in one line: the pendant program owns the motion, the PC only fills a "mailbox" (RTDE registers) with the latest offset, and a thread in the pendant program copies that mailbox into `path_offset_set()` every 2 ms.

Scope: small corrections, typically a few millimetres and a few degrees. Large changes to the path itself are out of scope and need re-planning instead.

System architecture



System architecture · PC, RTDE registers, pendant program

The main program and the offset thread run side by side inside one pendant program; the controller adds their results every 2 ms. The PC never sends motion commands, only offset numbers.

RTDE register mapping

The PC writes eight input registers; the pendant thread reads them with `read_input_float_register()` and `read_input_integer_register()`. Registers 24–47 are the range intended for external RTDE clients, so they do not clash with fieldbus use.

RTDE field (PC writes)	URScript read (robot)	Meaning	Unit
`input_double_register_24`	`read_input_float_register(24)`	Offset X	m
`input_double_register_25`	`read_input_float_register(25)`	Offset Y	m

RTDE field (PC writes)	URScript read (robot)	Meaning	Unit
`input_double_register_26`	`read_input_float_register(26)`	Offset Z	m
`input_double_register_27`	`read_input_float_register(27)`	Offset Rx (rotation vector)	rad
`input_double_register_28`	`read_input_float_register(28)`	Offset Ry (rotation vector)	rad
`input_double_register_29`	`read_input_float_register(29)`	Offset Rz (rotation vector)	rad
`input_int_register_24`	`read_input_integer_register(24)`	Heartbeat counter, +1 every PC cycle	count
`input_int_register_25`	`read_input_integer_register(25)`	Offset valid flag: 1 = use, 0 = ignore	flag

Optional robot → PC feedback: `output_int_register_24` = 1 while the offset thread is active, so the PC knows when to start sending.

Sequence of operation

The program runs in four phases. Phases 2 and 3 run in parallel: the main program moves the robot while the thread and the PC keep the offset updated.

Phase 1: Setup (before motion)

1. PC connects to the robot over RTDE (port 30004) and registers its input recipe (registers 24–29 doubles, 24–25 ints).
2. PC writes a zero offset, valid flag = 0, and starts the heartbeat.
3. Operator starts the program on the teach pendant.
4. Pendant program sets limits: `path_offset_set_max_offset()` and `path_offset_set_alpha_filter()`.

5. Pendant program calls `path_offset_enable()`, then `path_offset_set([0,0,0,0,0,0], type)` once.
6. Pendant program starts the offset thread and sets `output_int_register_24 = 1`.

Phase 2: Motion (main program)

7. Main program runs the offline path as normal: `movel` / `movep` waypoints taught or imported on the pendant.

Phase 3: Live correction (in parallel with phase 2, every 2 ms)

8. Gocator scans the edge ahead of the tool.
9. PC computes the 6D deviation between the scanned edge and the nominal path, filters it, clamps it, and delays it so it applies when the tool reaches that spot.
10. PC writes the 6 offset values, valid flag = 1, and heartbeat + 1 to the RTDE registers.
11. Robot thread reads the registers. If the heartbeat moved and valid = 1, it calls `path_offset_set(offset, type)`; otherwise it holds the last offset or ramps it to zero.
12. Controller adds the offset to the current point of the offline path and moves the robot to the corrected pose.

Phase 4: Shutdown

13. Main program finishes the last waypoint.
14. Thread is stopped (`kill`), offset ramps out with `path_offset_disable()`.
15. Pendant program sets `output_int_register_24 = 0`; PC stops sending.

Robot-side pseudocode (pendant program)

This is one PolyScope program: a Thread node plus the normal main program. In PolyScope it is built with a **Thread** node and **Script** nodes; it is shown here as URScript-style pseudocode. Offset type 2 applies the offset in the TCP (tool) frame, as in UR's own example.

```
# ---------- Settings ----------
OFFSET_TYPE   = 2          # 2 = TCP frame
MAX_TRANS     = 0.003      # 3 mm max shift
MAX_ROT       = 0.05       # ~3 deg max rotation
ALPHA         = 0.27       # smoothing, tune on real robot
TIMEOUT_TICKS = 25         # 25 x 2 ms = 50 ms without new data
```

```
# ---------- Offset thread (runs in parallel) ----------
thread offsetThread():
  offset     = [0,0,0,0,0,0]
  lastBeat   = read_input_integer_register(24)
  staleTicks = 0
  while True:
    beat  = read_input_integer_register(24)
    valid = read_input_integer_register(25)

    if beat != lastBeat and valid == 1:      # fresh data from PC
      offset = [read_input_float_register(24),
                read_input_float_register(25),
                read_input_float_register(26),
                read_input_float_register(27),
                read_input_float_register(28),
                read_input_float_register(29)]
      lastBeat   = beat
      staleTicks = 0
    else:
      staleTicks = staleTicks + 1
      if staleTicks > TIMEOUT_TICKS:         # PC silent: fade out
        offset = offset * 0.98               # slow ramp to zero
      end
    end

    path_offset_set(offset, OFFSET_TYPE)
    sync()                                   # wait one 2 ms cycle
  end
end

# ---------- Main program ----------
path_offset_set_max_offset(MAX_TRANS, MAX_ROT)
path_offset_set_alpha_filter(ALPHA)
path_offset_enable()
path_offset_set([0,0,0,0,0,0], OFFSET_TYPE)  # set once before use

thrd = run offsetThread()
write_output_integer_register(24, 1)        # tell PC: ready

movej(approach_pose)
movel(wp_1)                                  # offline path, unchanged
movel(wp_2)
...
```

```
movel(wp_N)

kill thrd
path_offset_disable()                          # ramps offset out smoothly
write_output_integer_register(24, 0)         # tell PC: done
```

Notes: `sync()` makes the loop run exactly once per 2 ms controller cycle. The `* 0.98` fade is pseudocode for element-wise scaling; in real URScript scale each of the six values. Call `path_offset_set()` once right after enabling; users report `path_offset_get()` fails until an offset has been set.

PC-side pseudocode: two separate modules

The PC software is split into two modules that run in their own threads and share only two small objects: the latest offset (`OffsetMailbox`) and the latest robot state (`RobotState`).

- **Module A, offset estimation (control):** sensor, nominal path, geometry, filtering, limits. It knows nothing about RTDE.
- **Module B, RTDE communication:** connection, registers, heartbeat, 2 ms timing. It knows nothing about sensors or geometry.

Where RTDE runs: the **RTDE server** runs inside the UR10e controller (control box, port 30004). The **RTDE client** is Module B on the PC. The teach pendant is only the screen for the program; nothing RTDE-related runs on it. Use UR's RTDE client library or `ur_rtde`'s `RTDEIOInterface` ; do **not** use `RTDEControlInterface` , because it uploads its own script and stops the pendant program.

Shared objects (the only link between A and B)

```
class OffsetMailbox:               # thread-safe, keeps only the newest value
    offset = [0,0,0,0,0,0]         # x,y,z in m; rx,ry,rz in rad (TCP frame)
    valid  = 0
    stamp  = 0                     # time the offset was computed
    put(offset, valid)             # written by Module A
    get() -> (offset, valid, stamp)   # read by Module B

class RobotState:                  # thread-safe
    tcp_pose    = None
    robot_ready = 0
    update(rtde_state)             # written by Module B
    latest() -> (tcp_pose, robot_ready)   # read by Module A
```

Module A: offset estimation (control)

```
# Knows: Gocator, nominal path, geometry, limits. Does NOT know RTDE.
def estimation_loop(mailbox, robot_state):
    last_off = [0,0,0,0,0,0]
    while running:
        scan = gocator.wait_next_profile()          # runs at sensor rate
        tcp, ready = robot_state.latest()
        if not ready:
            mailbox.put([0,0,0,0,0,0], valid=0)
            continue

        if scan.ok:
            raw    = compute_6d_deviation(scan, nominal_path, tcp)
            delay  = lookahead_distance / tool_speed  # scanner is ahead
            due    = delay_buffer.push(raw, delay)
            smooth = low_pass(due)
            rate   = limit_rate(smooth, max_step_per_cycle)
            off    = clamp(rate, MAX_TRANS, MAX_ROT)
            mailbox.put(off, valid=1)
            last_off = off
        elif scan_gap_short():
            mailbox.put(last_off, valid=1)          # hold last good value
        else:
            mailbox.put(last_off, valid=0)          # robot will fade out
```

Module B: RTDE communication (client)

```
# Knows: robot IP, registers, heartbeat, timing. Does NOT know sensors.
def rtde_loop(mailbox, robot_state):
    rtde = RTDE_connect(robot_ip, port=30004)
    rtde.setup_input_recipe([
        input_double_register_24 .. input_double_register_29,  # 6D offset
        input_int_register_24,                                 # heartbeat
        input_int_register_25 ])                               # valid flag
    rtde.setup_output_recipe([ output_int_register_24, actual_TCP_pose ])
    rtde.start()

    beat = 0
    write_registers(rtde, [0,0,0,0,0,0], valid=0, beat=beat)

    while running:
        state = rtde.receive()                    # paces loop at 2 ms
```

```
        robot_state.update(state)                    # share pose + ready

        off, valid, stamp = mailbox.get()
        if state.output_int_register_24 != 1:        # robot not ready
            off, valid = [0,0,0,0,0,0], 0
        if now() - stamp > STALE_LIMIT:              # Module A stopped
            valid = 0

        beat = beat + 1
        write_registers(rtde, off, valid, beat)

    write_registers(rtde, [0,0,0,0,0,0], valid=0, beat=beat + 1)
    rtde.stop()
```

Start-up

```
mailbox     = OffsetMailbox()
robot_state = RobotState()
start_thread(rtde_loop,       mailbox, robot_state)
start_thread(estimation_loop, mailbox, robot_state)
```

Why the split helps: Module A can be tested offline with recorded scans and no robot; Module B can be swapped (for example back to a socket) without touching the control code; and each runs at its own rate (sensor rate vs 500 Hz), with the mailbox always holding the newest value. Units must match the robot: metres for X, Y, Z and radians (rotation vector) for Rx, Ry, Rz, in the same frame as `OFFSET_TYPE` on the robot.

Safety, limits and tuning

The offset is limited three times: on the PC (clamp + rate limit), on the robot (`path_offset_set_max_offset`), and by the alpha filter. A bad sensor reading can therefore never push the tool more than the set limit.

- **Size limit:** start with 3 mm and 0.05 rad (~3°). Raise only after testing.
- **Rate limit:** cap how much the offset may change per 2 ms cycle so the wrist does not jerk, especially for rotations.
- **Alpha filter:** UR reports 0.27 was enough on a UR10 in their example; the right value depends on payload, TCP, speed and how fast the offset changes. Tune on the real robot.

- **Lookahead delay:** the Gocator sees the edge before the nozzle reaches it. Delay each offset by distance ÷ speed, or the correction lands too early.
- **Data loss:** if the heartbeat stops, hold the last value briefly, then ramp to zero. Never jump to zero.
- **Frame choice:** TCP frame (type 2) for corrections relative to the nozzle; base frame for fixed world shifts. Rotations in TCP frame turn around the tool tip.
- **Reach and singularities:** 6D rotations move the wrist. Check that the offline path keeps margin from joint limits and wrist singularities.
- **Test order:** URSim first, then the real robot at low speed with no sealant, then full speed.

Open question: confirm the PolyScope version on your UR10e is 5.6.0 or newer (path offset was released in 5.6.0).

Socket TCP/IP vs RTDE

Only the data link changes. The pendant program, the offline path and `path_offset_set()` stay the same; what changes is how the thread gets the numbers.

Aspect	Socket TCP/IP (last year)	RTDE (this approach)
Who drives the exchange	Robot program calls `socket_open()` / `socket_read_ascii_float()`	Controller updates registers itself, independent of the program
Timing	Irregular, whenever the PC sends	Fixed, up to 500 Hz, locked to the controller cycle
If PC is late	Read waits or times out; thread stalls; offset jumps when data returns	Register keeps last value; thread never waits
Data format	Text string, must be parsed (" (0.001,0.002,...)")	Typed doubles and ints, no parsing
Robot-side code	Open socket, read, check count, handle errors	One `read_input_float_register()` per value
Data loss detection	Must design your own	Heartbeat register, simple to add

Aspect	Socket TCP/IP (last year)	RTDE (this approach)
Feedback to PC	Separate send code	Output registers + robot state in same connection
Setup effort	Very simple to start	Needs an input recipe on the PC

What to change when moving from socket to RTDE:

1. **Robot:** replace `socket_open` / `socket_read_ascii_float` in the thread with `read_input_float_register(24…29)` and the heartbeat check. Keep `path_offset_set()` as is.
2. **PC:** replace the TCP send loop with an RTDE input recipe and register writes at 500 Hz.
3. **Remove** string formatting and parsing on both sides.

Advantages of RTDE and common misconceptions

RTDE gives smoother correction because the robot never waits for the PC; it always reads the latest value once per 2 ms cycle.

Advantages

- **Deterministic timing:** updates are locked to the 500 Hz controller cycle, so the offset changes evenly.
- **Non-blocking:** a late PC packet never stalls the thread or the motion.
- **Less code, fewer bugs:** typed values, no string parsing, no socket error handling in URScript.
- **Built-in health check:** a heartbeat register makes data loss easy to detect.
- **Two-way in one link:** the PC can read robot state (TCP pose, program status) on the same connection for logging and sync.
- **Pendant program untouched:** the offline path stays on the teach pendant, editable by operators as before.

Misconception: "RTDE cannot work with an offline path running on the pendant."

This is not correct. RTDE registers are read by the pendant program like any other variable, while the program runs. The confusion usually comes from one of these:

- **Using `RTDEControlInterface` (ur_rtde):** it uploads its own control script, which replaces the pendant program. Use only register writes.

- **Mixing up two methods:** streaming every pose with `servoj` from a PC does need the path on the PC. Path offset does not.
- **Reading the offset in the main program:** it then updates only between moves. Read it in a thread.
- **Register conflict:** another client already uses the same registers. Use free ones in 24–47.

Sources

Path offset (robot side)

- path_offset_set(offset, type), UR Script Manual: offset applied every 500 Hz cycle, thread example with `sync()`, frame types 1–4.
- Modify robot trajectory by overlaying custom motion, Universal Robots: Online Path Offset since PolyScope 5.6.0, separate thread over an existing motion, alpha filter.
- URScript Script Directory, path_offset functions, Universal Robots: reference for enable, disable, get, set, alpha filter and max offset.

RTDE (PC side)

- Real-Time Data Exchange (RTDE) Guide, Universal Robots: registers 24–47 reserved for external RTDE clients.
- RTDE Guide, article version, Universal Robots: RTDE client can run on the control box or any external PC.
- RTDE manual, PolyScope 10.10, Universal Robots: heavy processing such as image processing belongs on an external PC.

Linking RTDE registers to a URScript loop

- Resolving UR RTDE register range conflicts, Industrial Monitor Direct (third party): URScript reading `read_input_float_register(24…29)` from an RTDE client.

Known issues (forum)

- Working with path_offset_set, UR Forum: users report type 2 offsets following the flange rather than an off-centre TCP; workaround script shared.
- Weaving: angular orientation, tool flange vs TCP, UR Forum: UR engineer workaround for the TCP-frame issue and bug report. Test this with the sealant nozzle before relying on type 2.

- path_offset_get not working as expected, UR Forum: set an offset once after enabling before calling `path_offset_get()`.